REPORT

Name: Nikhil Verma

Roll No: 202310101200035

Github Repo : Link

Chat Application

Architecture:

I built a client–server chat application. The server runs on one main thread whose job is to accept incoming connections. Whenever a new client connects, the server creates a new thread to handle that client. Each connected user gets their own thread, which makes it possible for multiple people to chat at the same time.

What the Server Keeps Track Of:

- **Clients list**: Stores all connected client socket objects.
- **Nicknames list**: Stores the usernames of connected users
- **Message history**: Stores the last 10 messages so that new users can see recent chat.
- **Lock**: One lock is used so that only one thread can access shared data at a time.

When a user sends a message, the server adds a timestamp to it and then broadcasts it to all connected clients. Commands such as **/quit**, **/users**, and **/help** are handled directly by the server.

Client Side:

The client program is simpler and uses two threads:

- **Main thread**: Reads the message typed by the user and sends it to the server
- **Receive thread**: Continuously listens for messages from the server and displays them

This design allows the user to send and receive messages at the same time without blocking.

Concurrency:

The application uses a **one-thread-per-client** model because it is simple and Python supports it easily.

```
thread = threading.Thread(
    target=self.handle_client, args=(client, address), daemon=True)
thread.start()
```

Daemon threads are used so that they automatically stop when the server shuts down.

Handling Shared Data:

The biggest challenge was multiple threads accessing the same lists at the same time. Without protection, this leads to **race conditions**, where the client list and nickname list become mismatched and the program crashes.

To avoid this, a **lock** is used whenever shared data is modified.

```
with self.lock:
    self.clients.append(client)
    self.nicknames.append(nickname)
```

For broadcasting messages, the server first copies the client list while holding the lock, and then releases the lock before sending messages. This prevents the lock from being held during slow network operations.

Testing:

The application was tested with multiple clients running at the same time. Different scenarios were tested such as:

- Users joining and leaving randomly
- Duplicate nickname attempts
- Disconnecting clients during chat
- Sending commands during conversations

The system performed well during testing:

- No crashes caused by race conditions
- Proper handling of client disconnections
- Commands worked as expected

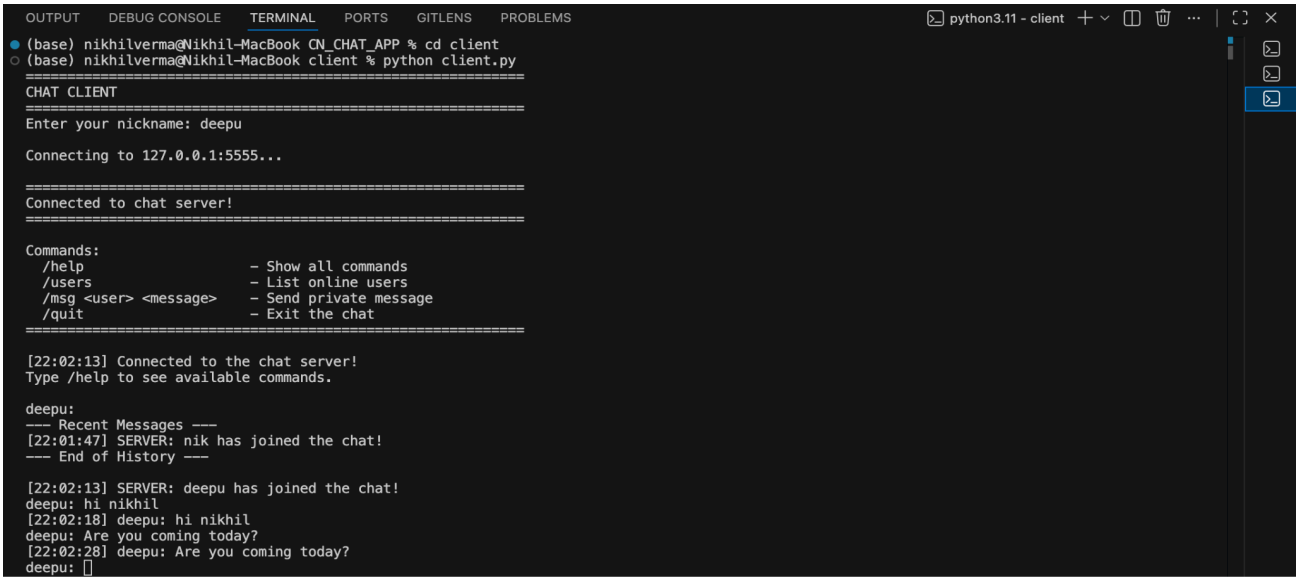
Terminal Output (Example Run):

Server output:

```
OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   PROBLEMS                    python3.11 - server

(base) nikhilverma@Nikhil-MacBook CN_CHAT_APP % cd server
(base) nikhilverma@Nikhil-MacBook server % cd python server.py
cd: string not in pwd: python
(base) nikhilverma@Nikhil-MacBook server % python server.py
============================================================
CHAT SERVER STARTED
============================================================
Host: 127.0.0.1
Port: 5555

Press Ctrl+C to stop the server
============================================================
[22:01:19] [LISTENING] Server is listening on 127.0.0.1:5555
[22:01:47] [CONNECTED] nik connected from 127.0.0.1:55370
[22:02:13] [CONNECTED] deepu connected from 127.0.0.1:55405
[22:02:18] [MESSAGE] deepu: hi nikhil
[22:02:28] [MESSAGE] deepu: Are you coming today?
```

Client output:

```
OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   PROBLEMS                    python3.11 - client

(base) nikhilverma@Nikhil-MacBook CN_CHAT_APP % cd client
(base) nikhilverma@Nikhil-MacBook client % python client.py
============================================================
CHAT CLIENT
============================================================
Enter your nickname: nik

Connecting to 127.0.0.1:5555...

============================================================
Connected to chat server!
============================================================

Commands:
  /help                    - Show all commands
  /users                   - List online users
  /msg <user> <message>    - Send private message
  /quit                    - Exit the chat
============================================================

[22:01:47] Connected to the chat server!
Type /help to see available commands.

[22:01:47] SERVER: nik has joined the chat!
[22:02:13] SERVER: deepu has joined the chat!
[22:02:18] deepu: hi nikhil
[22:02:28] deepu: Are you coming today?
nik:
```



Challenges:

Race Conditions: The program crashed when multiple users connected at the same time. The client list and nickname list became unsynchronized. This was solved by updating both lists together inside a lock and making copies before iterating.

Dead Connections: The server sometimes crashed when trying to send messages to a closed socket. This was fixed using try–except blocks and automatically removing disconnected clients.

Port Already in Use: While restarting the server quickly, an address already in use error occurred because the port was still occupied by the previous process.

Final Thoughts:

Threading turned out to be more complex than expected.To keep the design simple, the application uses **one lock and one thread per client**, which worked well for this project and handled multiple users smoothly.